

Probeklausur Systemprogrammierung 2024/2025 — Lösungen

Beachte: Unterschied zwischen Pseudocode und C. Verwenden Sie für die Beantwortung möglichst den eingeplanten Platz.

Aufgabe 1.) Datentypen (1 + 2 + 1 Punkte)

Lösung:

1.1 Weshalb wird empfohlen `stdint.h` bzw. `inttypes.h` zu inkludieren und diese Typen zu verwenden?

Lösung:

- `stdint.h` liefert fest breite Ganzzahltypen (z. B. `int8_t`, `uint16_t`, `int32_t`, `uint64_t`) — dadurch ist der Code portabler und das Verhalten (Größen) ist auf allen Plattformen eindeutig.
- `inttypes.h` ergänzt Format-Makros (z. B. `PRId32`, `PRId64`) für `printf/scanf`, damit korrektes Drucken/Einlesen plattformunabhängig gelingt.
- Vermeidet Fehler durch Annahmen über die Größe von `int`, `long` etc., wichtig bei Bitmanipulation, Binärformaten, Netzwerkprotokollen und hardware-nahen Anwendungen.

1.2 Was ist Padding? Unter welchen Umständen hat Padding in C einen Einfluss auf die Programmierung?

Lösung:

- Padding sind automatisch eingefügte Füllbytes innerhalb von Strukturen, damit Felder die vom Typ geforderte Alignment-Grenzen erfüllen.
- Einfluss: ändert `sizeof(struct)`, beeinflusst Speicherlayout (wichtig bei Binärformaten, Speichermapping, Serialisierung, Netzwerkprotokollen), kann zu unerwarteter Speicherbelegung oder Performance-Unterschieden führen. Auch bei ABI-Kompatibilität zwischen Compilern/Plattformen kritisch.

1.3 Was ist Alignment und wie hängt es mit Padding zusammen?

Lösung:

- Alignment ist die Anforderung, dass ein Datentyp auf einer Adresse startet, die ein Vielfaches eines bestimmten Werts ist (z. B. 4-Byte-Alignment für 32-bit-Felder).
- Padding wird eingefügt, damit nachfolgende Felder die nötigen Alignment-Grenzen erreichen. Somit ist Padding die Folge von Alignment-Anforderungen.

Beispiel (kurz):

```
struct A { // typisches Layout auf 32-bit
    uint8_t a; // offset 0
    // 3 bytes padding
    uint32_t b; // offset 4
};
```

Aufgabe 2.) Lieblingsfarbe (3 Punkte)

Lösung:

- struct mit 8-bit Komponenten und union für 32-Bit Zugriff. Hinweis: der numerische raw-Wert hängt von Endianness ab.

Beispielimplementierung in C:

```

#include <stdio.h>
#include <stdint.h>

typedef struct {
    uint8_t r;
    uint8_t g;
    uint8_t b;
    uint8_t a; // optional (z.B. Alpha oder Padding)
} RGBColor;

typedef union {
    RGBColor col;
    uint32_t raw;
} ColorUnion;

void printColor(const ColorUnion *c) {
    // Ausgabe: RGB-Komponenten und raw-Wert in hex
    printf("Lösung: R=%u G=%u B=%u A=%u, raw=0x%08X\n",
        (unsigned)c->col.r, (unsigned)c->col.g,
        (unsigned)c->col.b, (unsigned)c->col.a,
        (unsigned)c->raw);
}

/* Beispielnutzung:
int main(void) {
    ColorUnion c;
    c.col.r = 10; c.col.g = 20; c.col.b = 30; c.col.a = 0xFF;
    printColor(&c);
    return 0;
}
*/

```

Hinweis: raw spiegelt die Byte-Reihenfolge des Hosts (Little-/Big-Endian). Wenn ein spezifisches Byte-Layout für Protokolle gebraucht wird, Byteordnungsfunktionen verwenden.

Aufgabe 3.) Bitmaskierung (2 + 4 Punkte)

Lösung:

3.1 Unterschiede zwischen Bitmaskierung (Masken in Ganzzahltypen) und Bitfeldern (C-Bitfields):

Lösung:

- Bitmaskierung:
 - Felder werden als Ganzzahl (z. B. `uint8_t`/`uint16_t`) gespeichert. Zugriff erfolgt mit AND/OR/SHIFT. Portabel, explizit und effizient.
 - Sehr kontrollierbar (exakte Bit-Positionen).
 - Beispiel: Flags in einem Byte.
- Bitfields (C):
 - Syntax: `unsigned flag:1;` in struct. Compiler legt Bit-Layout fest — kann von Compiler/Platform variieren (Endianness, Packing).
 - Bequem und lesbar, aber weniger portabel für Binärformate oder Interoperabilität.
 - Können Optimierungen ermöglichen, jedoch ist die genaue Position der Bits nicht garantiert.

Beispiele (analog):

Bitmaskierung:

```
// Bitmasken-Variante
#define FLAG_READ  (1u << 0)
#define FLAG_WRITE (1u << 1)
#define FLAG_EXEC  (1u << 2)

uint8_t flags = 0;
flags |= FLAG_READ;           // setzen
if (flags & FLAG_WRITE) { ... } // prüfen
flags &= ~FLAG_EXEC;         // löschen
```

Bitfields:

```
// Bitfield-Variante
typedef struct {
    unsigned read  : 1;
    unsigned write : 1;
    unsigned exec  : 1;
    unsigned reserved : 5;
} FlagsBF;

FlagsBF f = { .read = 1, .write = 0, .exec = 0 };
if (f.write) { ... }
```

3.2 Paritätsbit-Funktion (gerade Parität).

Lösung: Die Funktion nimmt ein 8-Bit-Byte (Daten) und berechnet das Paritätsbit, das gesetzt werden muss, damit die Gesamtanzahl gesetzter Bits (Datenbits plus Paritätsbit) gerade ist.

Implementierung in C:

```
#include <stdint.h>

/* Gibt 0 oder 1 zurück: das Paritätsbit für gerade Parität.
   Wenn die Anzahl der Einsen in 'byte' gerade ist, ist Paritätsbit==0.
   Wenn ungerade, muss Paritätsbit==1, damit Gesamtanzahl gerade wird.
*/
uint8_t even_parity_bit(uint8_t byte) {
    // __builtin_popcount liefert Anzahl Einsen; portable Alternative weiter unten
    unsigned ones = __builtin_popcount((unsigned)byte);
    return (ones % 2) ? 1u : 0u;
}

/* Alternative ohne builtin:
uint8_t even_parity_bit(uint8_t byte) {
    uint8_t v = byte;
    v ^= v >> 4;
    v ^= v >> 2;
    v ^= v >> 1;
    uint8_t parity = v & 1; // 1 = ungerade Anzahl Einsen
    return parity ? 1u : 0u;
}
*/
```

Aufgabe 4.) Dynamische Speicherverwaltung (5 Punkte)

Lösung:

- Implementiere `mem_get_Sizes(...)`, die Heap-Blöcke im globalen Array `gHeap` durchwandert und die insgesamt belegten bzw. freien Bytes (inkl. Header-Größen) zählt.
- Annahmen/Robustheit: Falls ein Block länger ist, als noch bytes im Heap verbleiben, wird der verbleibende Bereich als Rest gezählt und Traversal beendet. Checksumme wird ignoriert.

Header-Definition und Funktion:

```
#include <stdint.h>
#include <stddef.h>

#define HEAPSIZE 1023
uint8_t gHeap[HEAPSIZE];

#pragma pack(push,1)
typedef struct {
    uint8_t flags;    // 0x01: allocated
    uint8_t checksum; // XOR-Checksum, ignoriert beim Zählen
    uint16_t length;  // Anzahl Datenbytes
} Header;
#pragma pack(pop)

#define HEADER_SIZE (sizeof(Header))

/* mem_get_Sizes: traversiert gHeap von Anfang bis HEAPSIZE
   und summiert allocatedBytes und freeBytes (jeweils inkl. Header).
   safe: falls ein Block unplausible Länge hat, wird der Rest heap als ein Block gezählt.
*/
void mem_get_Sizes(size_t *allocatedBytes, size_t *freeBytes) {
    size_t alloc = 0;
    size_t freeb = 0;
    size_t pos = 0;

    while (pos < HEAPSIZE) {
        // Prüfe, ob genug Platz für Header bleibt
        if (pos + HEADER_SIZE > HEAPSIZE) {
            // Unvollständiger Header am Ende: verbleibende Bytes als free zählen
            freeb += (HEAPSIZE - pos);
            break;
        }

        Header hdr;
        // Header aus gHeap lesen (Little/Big-Endian irrelevant, length ist Host-orientiert hier)
        hdr.flags = gHeap[pos];
        hdr.checksum = gHeap[pos + 1];
        // length: stored as 2 bytes; umkopieren
        hdr.length = (uint16_t)gHeap[pos + 2] | ((uint16_t)gHeap[pos + 3] << 8);

        size_t blockTotal = HEADER_SIZE + (size_t)hdr.length;

        // Validierung: wenn blockTotal über das Ende hinausgeht, beschränken
        if (pos + blockTotal > HEAPSIZE) {
            blockTotal = HEAPSIZE - pos; // zählen Rest und beenden danach
        }

        if (hdr.flags & 0x01u) {
```

```

        alloc += blockTotal;
    } else {
        freeb += blockTotal;
    }

    pos += blockTotal;
}

if (allocatedBytes) *allocatedBytes = alloc;
if (freeBytes)      *freeBytes = freeb;
}

```

Erläuterung:

- Wir kopieren Header-Bytes manuell, um Alignment/Endian-Probleme zu vermeiden.
- `length` wird hier als 16-Bit Little-Endian interpretiert (wie aus dem Beispiel: LSB zuerst). Falls das Format anders ist, Byte-Ordnung anpassen.
- Header-Größe (4 Bytes) wird stets dem Block zugerechnet.

Aufgabe 5.) Multitasking (2 + 2 + 4 + 2 + 2 Punkte)

Lösung:

5.1 Semaphor — Was ist ein Semaphor? P() und V()

Lösung:

- Ein Semaphor ist ein Synchronisationsprimitive zur Kontrolle des Zugriffs auf gemeinsame Ressourcen. Es kann einen Zähler haben (Counting Semaphore) oder binär sein (Binary Semaphore/Mutex-ähnlich).
- Operationen:
 - P() (auch Wait/Down/take): dekrementiert den Zähler; wenn der Zähler 0 ist, blockiert der aufrufende Thread/Task bis verfügbar.
 - V() (auch Signal/Up/give): inkrementiert den Zähler; wenn Tasks blockiert sind, wird einer (oder mehrere bei Counting) geweckt.
- Verwendung: Schutz von Ressourcen, Steuerung der Anzahl gleichzeitiger Zugriffe, Signalisierung zwischen Tasks (Producer/Consumer).

5.2 Semaphor in FreeRTOS — Erklärungen der Funktionen

Lösung:

- `SemaphoreHandle_t xSemaphoreCreateCounting(UBaseType_t uxMaxCount, UBaseType_t uxInitialCount);`
 - Erzeugt ein zählendes Semaphor mit maximalem Zähler `uxMaxCount` und initialem Zähler `uxInitialCount`.
 - Rückgabe: `SemaphoreHandle_t` (Handle zur Semaphore) oder `NULL` bei Fehler (z. B. Out-of-memory).
- `xSemaphoreGive(SemaphoreHandle_t xSemaphore);`
 - Erhöht den Zähler der Semaphore (signal).
 - Rückgabe: `BaseType_t` — `pdTRUE` wenn erfolgreich; bei besonderen Varianten (ISR-Version `xSemaphoreGiveFromISR`) gibt es spezielle Rückgabewerte für Kontextwechselbedarf.
- `xSemaphoreTake(SemaphoreHandle_t xSemaphore, TickType_t xTicksToWait);`
 - Versucht, die Semaphore zu nehmen (dekrementieren). Wenn nicht verfügbar, blockiert der Aufrufer bis `xTicksToWait` Ticks verstrichen sind (`portMAX_DELAY` = unendlich).
 - Rückgabe: `pdTRUE` bei Erfolg (erhalten), `pdFALSE` bei Zeitüberschreitung.

Hinweis: In FreeRTOS sind `BaseType_t` und `TickType_t` plattformabhängige typedefs; typische Werte: `pdTRUE`, `pdFALSE`.

5.3 Producer/Consumer — Implementation (Producer alle 30 ms produzieren, Consumer gibt alle 30 ms aus) Lösung (FreeRTOS-Pseudocode/C):

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include <stdio.h>

/* Einfaches Ereignis (z.B. ein int Event) */
typedef int Event;
#define QUEUE_LENGTH 10

static QueueHandle_t eventQueue = NULL;

void ProducerTaskFunc(void * pvParameters) {
    TickType_t delay = pdMS_TO_TICKS(30);
    Event ev = 0;
    for (;;) {
        ev++; // "Produzieren"
        xQueueSend(eventQueue, &ev, portMAX_DELAY);
        // Lösung: Warte 30ms bis zur nächsten Produktion
        vTaskDelay(delay);
    }
}

void ConsumerTaskFunc(void * pvParameters) {
    TickType_t wait = pdMS_TO_TICKS(30);
    Event ev;
    for (;;) {
        // Warten auf ein Ereignis - max 30ms blockieren
        if (xQueueReceive(eventQueue, &ev, wait) == pdTRUE) {
            printf("Lösung: Consumer empfangt Ereignis %d\n", ev);
        } else {
            // Timeout - optional printf oder Weiterverarbeitung
            printf("Lösung: Consumer: kein Ereignis innerhalb 30ms\n");
        }
    }
}

/* Setup (z.B. in main):
eventQueue = xQueueCreate(QUEUE_LENGTH, sizeof(Event));
xTaskCreate(ProducerTaskFunc, "Producer", 256, NULL, tskIDLE_PRIORITY+2, NULL);
xTaskCreate(ConsumerTaskFunc, "Consumer", 256, NULL, tskIDLE_PRIORITY+2, NULL);
vTaskStartScheduler();
*/
```

5.4 Scheduling — Skizze (System-Tick 10 ms) Lösung:

- Annahmen: Beide Tasks (Producer, Consumer) haben gleiche Priorität > Idle. Zeitscheibe = 10 ms (Round-Robin). Producer/Consumer blockieren jeweils 30 ms mit `vTaskDelay()` nach Ausführung.
- Timeline (vereinfacht, t in ms):

t = 0..10:

- Scheduler wählt z. B. Producer (Round-Robin). Producer läuft kurz (z. B. <10ms), sendet Event, ruft `vTaskDelay(30ms)` und blockiert.
- Rest der Zeitscheibe: Consumer wird ausgeführt falls bereit (wenn es Daten gibt in Queue). Dann Consumer liest und evtl. printf. Nach der Ausführung blockiert er für 30ms (oder bleibt bereit, je nach Implementation). t = 10..20:

- Falls beide blockiert, Idle-Task läuft (niedrige Priorität). $t = 20..30$:
- Idle oder eventuell andere Tasks. $t = 30$:
- Producer-Delay endet, Producer wird ready; Scheduler vergibt CPU im nächsten Takt.
- Cycle wiederholt.

Kurz: Producer/Consumer werden jeweils ca. alle 30ms aktiv, Idle bekommt CPU wenn alle anderen blockiert sind. Bei gleicher Priorität wird Round-Robin verwendet — jeder lauffähige Task bekommt Zeitscheiben von 10ms.

5.5 Producer/Consumer mit Produkten (struct Product)

Lösung:

- Geeignete Datenstruktur: eine Queue (FIFO) — in FreeRTOS `xQueueCreate(count, sizeof(Product))`. Die Queue kopiert das `Product` beim Senden in den internen Speicher; für große Produkte empfiehlt sich Pointer-Queue mit Puffer-Pool oder statische Speicher-Pools.

Beispiel (Pseudocode / FreeRTOS-C):

```
typedef struct {
    int id;
    char name[32];
    float price;
} Product;

#define PRODUCT_QUEUE_LENGTH 8
static QueueHandle_t productQueue;

void ProducerTaskFunc(void * pvParameters) {
    TickType_t delay = pdMS_TO_TICKS(30);
    Product p;
    int nextId = 0;
    for (;;) {
        // Erzeuge Produkt
        p.id = nextId++;
        snprintf(p.name, sizeof(p.name), "Prod-%d", p.id);
        p.price = 1.23f * p.id;

        // Senden (Kopie in Queue)
        if (xQueueSend(productQueue, &p, portMAX_DELAY) != pdTRUE) {
            // Fehlerbehandlung
        }

        vTaskDelay(delay);
    }
}

void ConsumerTaskFunc(void * pvParameters) {
    Product p;
    for (;;) {
        if (xQueueReceive(productQueue, &p, portMAX_DELAY) == pdTRUE) {
            // Verarbeitung
            printf("Lösung: Produkt empfangen: id=%d name=%s price=%.2f\n",
                p.id, p.name, p.price);
        }
    }
}

/* Setup:
productQueue = xQueueCreate(PRODUCT_QUEUE_LENGTH, sizeof(Product));
*/
```

Hinweis: Wenn `Product` groß ist, stattdessen `Queue` von `Product*` verwenden und Speicher-Management (Pools) sicherstellen, damit keine Speicherlecks oder Fragmentierung entstehen.
